

Month 6 – Phase 7 + 8: System Design + Interviews (Days 141-182) – Easy → Hard

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 4 Jan 2027 – 14 Feb 2027 (Days 141-182)

 **Your slots (IST):** Mon–Fri **7:30-8:30pm** & **9:00-10:00pm** · Sat–Sun **9:00am-12:00pm, 3:00-5:30pm, 6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import [calendar/mern-mastery-study.ics](https://calendar.google.com/calendar/ical/calendar/mern-mastery-study.ics)

Master guide: [19-phases-678-easy-to-hard-complete.md](#)

Roadmap: [ROADMAP.md](#)

Sub-phase map

Sub	Days	Difficulty	This PDF section
7A	141-143	● SD basics	WEEK 21 (Days 141-143)
7B	144-147	● Easy HLD	WEEK 21B (Days 144-147)
7C	148-152	● Medium HLD	WEEK 22 (Days 148-152)
7D	153-156	● Complex HLD	WEEK 22B (Days 153-156)
7E	157-160	● Distributed	WEEK 23 (Days 157-160)
7F	161-164	● Product HLD (YouTube/Drive/Typeahe ad)	WEEK 23B (Days 161-164)
7G	165-168	● CS + FE MC + company loops	WEEK 24 (Days 165-168)
8A-8E	169-182	● → ● Mocks + apply	WEEK 25-26 (Days 169-182)

Do NOT design Chat/Payment (7D) before URL Shortener (7B).
Do NOT read Kafka (7F) before Distributed (7E).

System Design Daily Task Template

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Use for **every** HLD exercise (45 min draw + 15 min explain aloud):

1. **Clarify requirements** — functional + non-functional (scale, latency, consistency).
 2. **Estimate** — DAU, QPS, storage (back-of-envelope).
 3. **API design** — key endpoints + payloads.
 4. **High-level diagram** — clients, LB, services, DB, cache, queue.
 5. **Data model** — core tables/documents + indexes.
 6. **Deep dive** — one hard part (hashing, fan-out, sharding).
 7. **Bottlenecks & tradeoffs** — what breaks at 10x scale.
 8. **Explain** — record yourself or teach a wall; 15 min max.
-

WEEK 21 – Phase 7A: SD building blocks (Days 141-143)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Theory first: [10c-system-design-prerequisites-complete.pdf](#) (complete before any HLD)

Problem statement

Study RADIO/RESCHED framework; complete 3 estimation drills: QPS for 10M DAU app, storage for photo upload service, bandwidth for video thumbnail CDN.

Learning objective

Structured interview opening; order-of-magnitude math.

Prerequisites

- [10-system-design-complete.md § Framework](#)

Step-by-step approach

1. Read framework section; note R=Requirements, A=Architecture, etc.
2. Drill 1: 10M DAU, 10 actions/day → avg QPS + peak 3x.
3. Drill 2: 1M users, 5 photos/day, 2MB avg → storage/year.
4. Drill 3: 100K concurrent viewers, 500kbps stream → egress.
5. Write formulas in [system-design/estimation-cheatsheet.md](#).

Exact Answer

- $10\text{M DAU} \times 10 \text{ actions/day} = 100\text{M actions/day} \approx 1,160 \text{ avg QPS}$, $\sim 3,500 \text{ peak QPS}$
- $1\text{M users} \times 5 \text{ photos/day} \times 2\text{MB} = 10\text{TB/day storage}$
- $100\text{K viewers} \times 500\text{kbps} = 50 \text{ Gbps egress}$

Complete Solution

Estimation Cheatsheet

- $\text{QPS} = (\text{DAU} \times \text{actions per user}) / 86400$; peak $\approx 3 \times \text{avg}$
- $\text{Storage} = \text{users} \times \text{items} \times \text{size} \times \text{retention period}$
- $\text{Bandwidth} = \text{concurrent users} \times \text{bitrate}$
- Always state assumptions explicitly in interviews

Expected output / acceptance criteria

- Cheatsheet with 3 worked examples and assumptions labeled.

Where to practice

- Theory: [10-system-design-complete.md](#)
- Grokking extras: <https://www.educative.io/courses/grokking-the-system-design-interview> (optional paid)

Interview connection

First 5 minutes of every HLD round — interviewers score structured thinking.

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 2 hours

Day 142 – Design: URL Shortener

Problem statement

Design bit.ly-like service: shorten, redirect, analytics optional, 100M URLs/month.

Learning objective

Hashing, base62, read-heavy caching, redirect latency.

Prerequisites

- [10-system-design-complete.md § Design 1 URL Shortener](#)

Step-by-step approach

1. Requirements: create short URL, 301/302 redirect, custom alias optional, TTL optional.
2. API: `POST /urls`, `GET /{code}` redirect.
3. ID: counter + base62 OR hash with collision retry.
4. Store: SQL or KV — code → long_url, user_id, created_at.
5. Cache hot codes in Redis; 99% reads.
6. Scale: DB sharding by code prefix; CDN edge for redirect optional discussion.

Hints

1. Read:write ratio ~100:1 favors aggressive caching.
2. 302 vs 301 affects analytics and SEO — state choice.
3. Unique constraint on short_code critical.

Exact Answer

- `POST /urls { longUrl } → { shortCode }`; `GET /{code} → 302 redirect`
- 100M URLs/month $\approx$ 40 writes/sec avg; reads 100 $\times$ higher $\rightarrow$ aggressive Redis cache
- Storage: $\sim 500 \text{ bytes} \times 100\text{M} = 50\text{GB/year}$

Complete Solution

Architecture

Client $\rightarrow$ LB $\rightarrow$ API Servers $\rightarrow$ Redis (hot codes) $\rightarrow$ PostgreSQL (code, long_url, user_i

Key Design Decisions

1. ID: *base62(counter) or hash with collision retry*
2. Cache: *cache-aside, 99% read hit rate target*
3. DB: *unique index on short_code; shard by prefix at scale*
4. Analytics: *async queue for click events*

Expected output / acceptance criteria

- One-page diagram + API list + storage schema in `system-design/url-shortener.md`.

Where to practice

- Theory section linked above
- Local: `mern-mastery/system-design/week-21/`

Common mistakes

- Jumping to microservices before single-service design.
- Ignoring collision handling for hash-based keys.

80 LPA Interview Twist

Netflix/Meta use Redis for rate limiting, pub/sub, and session store — expect distributed Redis cluster and failover discussion.

Interview connection

#1 beginner HLD problem; appears on virtually every company list.

Time estimate: 2 hours weekday + 15 min explain aloud

Day 143 – Paste Bin + Distributed Rate Limiter

Problem statement

Design Pastebin with TTL expiration; separate 30-min design for global rate limiter.

Step-by-step approach (Pastebin)

1. POST paste content → id; GET by id; expiry worker deletes old pastes.
2. Storage: object store for large text; metadata in DB.
3. Cron or TTL index in Redis for expiration.

Step-by-step approach (Rate Limiter)

1. Token bucket or sliding window log per user/IP.
2. Redis INCR + EXPIRE or Lua script atomicity.
3. Sync vs async rejection; 429 response headers.

Exact Answer

- `allowRequest(userId)` returns true if token available, false otherwise
- Tokens refill at configured rate up to max capacity

Complete Solution

```
class TokenBucketRateLimiter {
  private buckets = new Map<string, { tokens: number; lastRefill: number }>();

  constructor(private maxTokens: number, private refillRate: number) {}

  allowRequest(userId: string): boolean {
    const now = Date.now();
    let bucket = this.buckets.get(userId) ?? { tokens: this.maxTokens, lastRefill: 0 };
    const elapsed = (now - bucket.lastRefill) / 1000;
    bucket.tokens = Math.min(this.maxTokens, bucket.tokens + elapsed * this.refillRate);
    bucket.lastRefill = now;
    if (bucket.tokens < 1) { this.buckets.set(userId, bucket); return false; }
    bucket.tokens -= 1;
    this.buckets.set(userId, bucket);
    return true;
  }
}
```

Where to practice

- [10-system-design-complete.md](#)

80 LPA Interview Twist

Netflix/Meta use Redis for rate limiting, pub/sub, and session store — expect distributed Redis cluster and failover discussion.

Time estimate: 2 hours each topic (split across day)

Day 144 – Caching + Load Balancing Deep Dive

Problem statement

Study notes; redraw URL shortener adding LB layer, multi-region cache, cache invalidation strategy.

Step-by-step approach

1. Cache patterns: cache-aside, write-through, write-back — when each applies.
2. LB algorithms: round robin, least connections, consistent hash.
3. Update Day 142 diagram with Redis cluster + 2 app servers.

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

```
import request from "supertest";
import app from "../src/app";

describe("POST /api/auth/login", () => {
  it("returns 200 with token", async () => {
    const res = await request(app).post("/api/auth/login").send({ email: "test@tes
    expect(res.status).toBe(200);
    expect(res.body.accessToken).toBeDefined();
  });
});
```

80 LPA Interview Twist

Netflix/Meta use Redis for rate limiting, pub/sub, and session store — expect distributed Redis cluster and failover discussion.

Time estimate: 2 hours

Day 145 – DSA: 2 Mediums Timed

Problem statement

Two random mediums from weak patterns list; 35 min each, no hints first 15 min.

Where to practice

- <https://leetcode.com/problemset/>

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

```
# Vercel (frontend): vercel deploy --prod
# Render/Railway (API): connect repo, set env vars, deploy
# Verify: curl https://api.example.com/health
# Update README with live demo URLs
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 2 hours

Day 146 – Sat: Blind URL Shortener + 3 LC

Problem statement

Redraw URL Shortener from blank paper in 45 min; then 3 LeetCode mediums.

Expected output / acceptance criteria

- Blind design within 45 min without notes; self-score /25 rubric (requirements, API, diagram, scale, tradeoffs).

Exact Answer

- `POST /urls { longUrl } → { shortCode }`; `GET /{code} → 302 redirect`
- 100M URLs/month $\approx$ 40 writes/sec avg; reads 100× higher → aggressive Redis cache
- Storage: $\sim 500 \text{ bytes} \times 100\text{M} = 50\text{GB/year}$

Complete Solution

Architecture

Client → LB → API Servers → Redis (hot codes) → PostgreSQL (code, long_url, user_i

Key Design Decisions

1. ID: base62(counter) or hash with collision retry
2. Cache: cache-aside, 99% read hit rate target
3. DB: unique index on short_code; shard by prefix at scale
4. Analytics: async queue for click events

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 8 hours

Day 147 – Sun: Feed Systems Intro + 3 LC

Problem statement

Read fan-out on write vs fan-out on read; sketch Twitter feed hybrid approach; 3 LC mediums.

Where to practice

- [10-system-design-complete.md § Design 2 Feed](#)

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

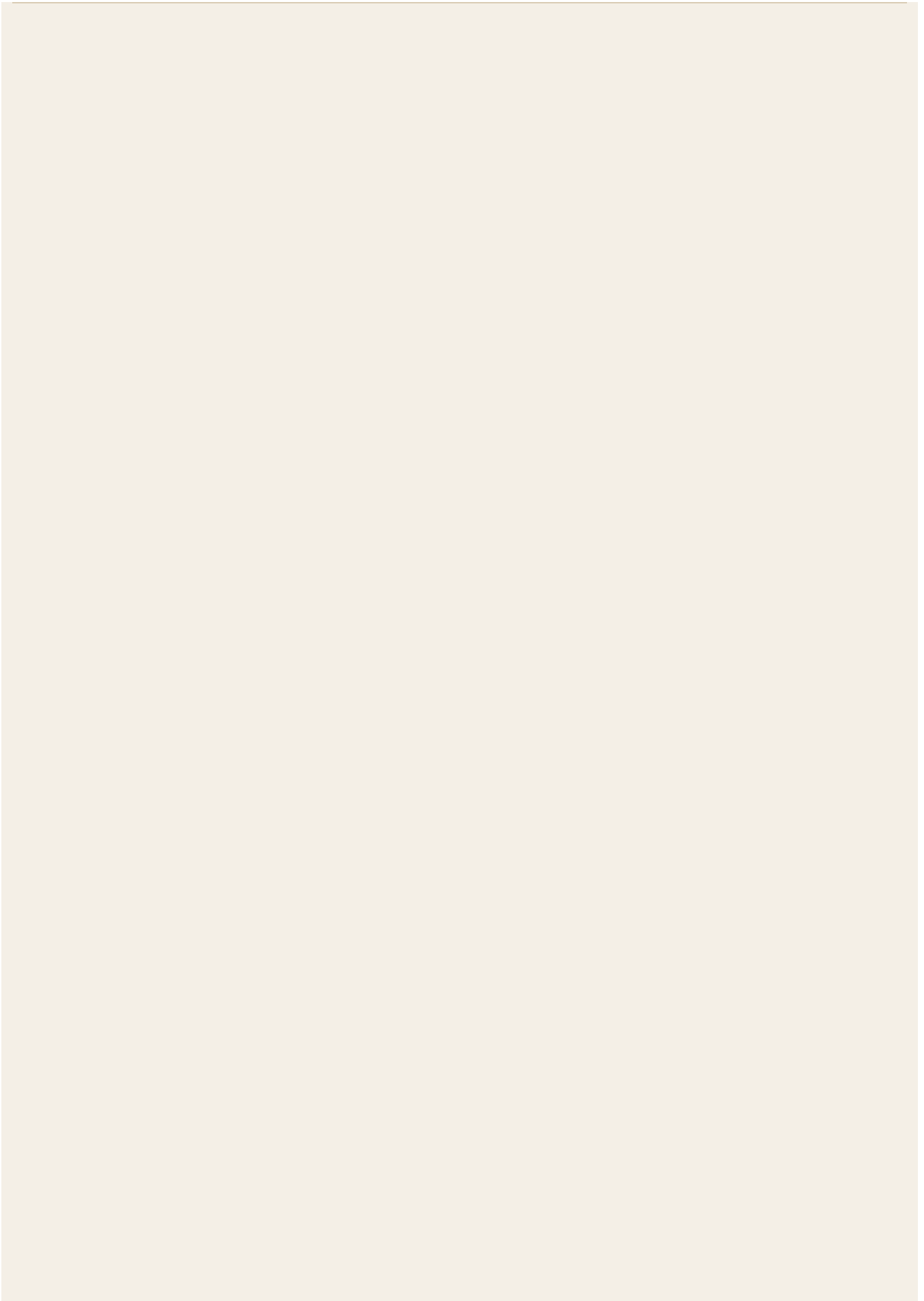
Mock Debrief Template

- DSA: /10 (approach, code, complexity, edge cases)
- HLD: /10 (requirements, estimation, diagram, deep dive, tradeoffs)
- Behavioral: /10 (STAR structure, specificity, result quantification)
- Communication: /10 (thinking aloud, clarifying questions, time management)
- Top 3 improvements for next mock

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 8 hours



WEEK 22 – Social + Notifications (Days 148-154)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Day	System	Key focus	Theory link
148	Twitter / X Feed	Fan-out, timeline merge	§ Design 2
149	Instagram	Media storage, CDN, likes	Same
150	Notification System	Queue, workers, preferences	§ Design 4
151	Kafka basics	Pub/sub, ordering, retention	Notes + YouTube summary
152	DSA 2 mediums	Timed	leetcode.com
153 Sat	Notification SD + 3 LC	Full doc	8h
154 Sun	Feed fan-out + 3 LC	Compare push vs pull	8h

Day 150 – Notification System (Detailed)

Problem statement

Design multi-channel notifications (push, email, SMS) for 50M users, 1B events/day.

Step-by-step approach

1. Ingest event API → Kafka topic partitioned by user_id.
2. Worker consumers route by user preferences table.
3. Template service; provider adapters (FCM, SendGrid, Twilio).
4. Idempotency keys dedupe retries.
5. Rate limit per user; priority queue for security alerts.

Exact Answer

- 1B events/day $\approx$ 12K events/sec avg; Kafka partitioned by user_id
- Workers route by user preferences to FCM/SendGrid/Twilio adapters
- Idempotency keys prevent duplicate notifications on retry

Complete Solution

Flow

Event API → Kafka (user_id partition) → Worker Pool → Template Service → Provider

Entities

- *UserPreferences*: { *userId*, *push*: bool, *email*: bool, *sms*: bool }
- *NotificationEvent*: { *id*, *userId*, *type*, *payload*, *idempotencyKey* }
- *Dead letter queue for failed deliveries after 3 retries*

Expected output / acceptance criteria

- Diagram with queue, workers, providers; failure handling documented.

Interview connection

Meta, Google, Amazon all ask notification or email pipeline variants.

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 2 hours + explain

WEEK 23 – Chat + Streaming (Days 155-161)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Day	System	Focus
155	WhatsApp Chat	WebSockets, message order, delivery receipts
156	Online Presence	Heartbeats, last seen, privacy
157	YouTube (lite)	Upload pipeline, transcoding, HLS
158	CDN + video storage	Edge caching, origin shield
159	DSA 2 mediums	
160 Sat	Chat SD + 3 LC	
161 Sun	Video SD + 3 LC	

Day 155 – Chat System (Detailed)

Problem statement

1:1 messaging, online status, message history, 10M concurrent connections discussion.

Prerequisites

- [10-system-design-complete.md § Design 3 Chat](#)

Step-by-step approach

1. WebSocket gateway sticky sessions; user → server mapping in Redis.
2. Send message: persist Cassandra/Scylla by conversation_id; push to recipient socket.
3. Delivery/read receipts async update.
4. History pagination cursor-based.
5. End-to-end encryption optional deep dive.

Exact Answer

- WebSocket gateway with sticky sessions; user→server mapping in Redis
- Messages persisted by conversation_id; cursor-based history pagination

- Delivery/read receipts updated asynchronously

Complete Solution

Components

- WS Gateway (sticky LB) → Message Service → Cassandra (*conversation_id partition*)
- Redis: *online presence, user→connection mapping*
- Send: *persist → push to recipient socket → ack receipt async*

Where to practice

- Local: [system-design/week-23/chat.md](#)

80 LPA Interview Twist

Netflix/Meta use Redis for rate limiting, pub/sub, and session store — expect distributed Redis cluster and failover discussion.

Time estimate: 2 hours weekday

WEEK 24 – E-commerce + Mock Interviews (Days 162-168)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Day	Focus
162	Uber / ride matching (basic geohash, dispatch)
163	E-commerce orders – § Design 5
164	Payment flow – idempotency, webhooks, ledger
165	8 STAR behavioral stories
166	Full mock: 1 DSA + 1 HLD + 1 behavioral
167 Sat	Apply 5 companies + resume
168 Sun	GitHub audit + LinkedIn + 220 LC milestone

STAR Stories Template (Day 165 – Write 8 Stories)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Complete one table row per story. **Memorize bullet points, not a script.**

#	Topic	Story file
1	Challenging bug you fixed	behavioral/01-bug.md
2	Disagreement with teammate	behavioral/02-disagreement.md
3	Tight deadline delivery	behavioral/03-deadline.md
4	Project you're most proud of	behavioral/04-proud-project.md
5	Learned something fast	behavioral/05-fast-learn.md
6	Failure and lesson	behavioral/06-failure.md
7	Improved performance	behavioral/07-performance.md
8	Mentored someone	behavioral/08-mentorship.md

STAR Template (Copy per story)

Story N: [Title]

Situation

- Team / company context (2 sentences)
- Scale or stakes (users, revenue, deadline)

Task

- Your specific responsibility (not the whole team's)

Action

- Step 1: [technical or leadership action]
- Step 2: ...
- Step 3: ...
- **I** statements – what YOU did

Result

- Quantified outcome (+40% perf, shipped 2 days early, zero sev-1 since)
- What you learned

60-Second Version

[Practice paragraph under 150 words]

Likely Follow-Up Questions

- What would you do differently?
- How did you measure success?

Story 1 Example Prompt – Challenging Bug

Problem statement

Write STAR for a production bug: symptoms, investigation, root cause, fix, prevention.

Step-by-step approach

1. Pick TaskFlow or ShopLite real bug (or realistic scenario).
2. Situation: production checkout failing 5% of requests.
3. Task: you owned API debugging.
4. Action: logs → reproduce → race condition in Redis cart merge → integration test added.
5. Result: error rate 0%; monitoring alert added.

Exact Answer

- Complete STAR story with quantified result (+5% → 0% error rate)

- 60-second version under 150 words, ready to recite aloud

Complete Solution

Story 1: Production Checkout Bug

****Situation:**** ShopLite checkout failed 5% of requests during a sale event, affecting revenue.

****Task:**** I was the backend owner responsible for diagnosing and fixing the Express checkout issue.

****Action:****

1. Correlated 5xx spikes with Redis timeout logs in application monitoring
2. Reproduced race condition: concurrent cart merge + checkout without atomic lock
3. Implemented Redis WATCH/MULTI/EXEC for atomic cart read-modify-write
4. Added Supertest integration test simulating 50 concurrent checkout requests
5. Deployed fix with feature flag and monitored error rate for 2 hours

****Result:**** Checkout error rate dropped from 5% to 0% within 24 hours. Zero checkout failures.

****60-Second Version:**** During a flash sale, 5% of ShopLite checkouts failed. I traced the issue to a race condition in the Redis cart management logic.

****Likely Follow-Ups:****

- "What would you do differently?" → Load test before sale events; chaos engineering
- "How did you measure success?" → Error rate metric in Datadog, customer support tickets

Expected output / acceptance criteria

- 8 files filled; each has 60-second version under 150 words.
- Practice aloud — under 2 min per story.

Interview connection

Amazon LP, Microsoft "tell me about a time", startup culture fit rounds.

80 LPA Interview Twist

Amazon LP interviews use STAR with 'I' statements and dig 3 levels deep on YOUR specific contribution — team answers score poorly. Prepare 2-minute and 30-second versions of each story.

Time estimate: 4 hours for all 8

Application Tracker (Days 167+ – Apply 5-8/week)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Maintain `applications/tracker.md` or spreadsheet:

Company	Role	Applied Date	OA	Phone	Onsite	Offer	Notes	Referral
Adobe	Full Stack MERN							
Salesforce	MTS							
Microsoft	SDE-2							
Google	L4							
Amazon	SDE-2							
Atlassian	Backend							
Uber	Full Stack							
Goldman Sachs	Engineer							

Application Task – Step-by-Step (Day 167)

1. Update resume PDF — one page, metrics from TaskFlow/ShopLite.
2. Tailor headline + top bullet per company job description (keywords).
3. Submit application before noon local; log in tracker immediately.
4. LinkedIn connect with recruiter + one engineer (personalized note, no pitch novel).
5. GitHub: pin TaskFlow, ShopLite; README badges green CI, live demo links.
6. Track rejection reasons to refine weak areas weekly.

Where to practice

- LinkedIn Jobs: <https://www.linkedin.com/jobs/>

- Company career pages: <https://careers.adobe.com/>, <https://www.metacareers.com/> (adjust per target)
- Levels.fyi interview reports: <https://www.levels.fyi/>

80 LPA Interview Twist

Flagship project reviews at 80 LPA focus on real-time consistency, conflict resolution, and horizontal scaling of Socket.io with Redis adapter.

Time estimate: 8 hours (Day 167 Saturday)

Day 166 – Full Mock Interview

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Problem statement

Simulate 3-round loop: 45 min DSA medium, 45 min HLD (draw), 30 min behavioral (2 STAR stories).

Step-by-step approach

1. Pick unseen medium from LeetCode random.
2. Pick system not designed this week (e.g., Ticketmaster).
3. Use Pramp or record split-screen.
4. Debrief: what broke, time management, communication fillers.

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

GitHub Audit Checklist

- [] Pin TaskFlow + ShopLite repos
- [] README: demo GIF, architecture diagram, env setup
- [] No secrets in git history
- [] CI badge green
- [] LinkedIn headline + featured demos updated
- [] LeetCode ≥ 220 verified

Where to practice

- Pramp: <https://www.pramp.com/>
- Interviewing.io (paid): <https://interviewing.io/>

Expected output / acceptance criteria

- Debrief notes in `mock-interviews/2026-XX-XX.md` with scores /10 per dimension.

Time estimate: 3 hours

Day 168 – GitHub Audit + LinkedIn + 220 LC

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Problem statement

Public profile ready for recruiter review; LeetCode count ≥ 220 .

Step-by-step approach

1. GitHub audit checklist:

- ☐ Pinned repos (max 6) — best projects first
- ☐ Each README: demo link, screenshot, architecture, tech stack
- ☐ No secrets in git history (`git log -p` spot check)
- ☐ Consistent commit messages last 90 days
- ☐ Green CI badges where applicable

2. LinkedIn:

- ☐ Headline: role + stack + impact
- ☐ About: 3 paragraphs + project links
- ☐ Featured section: live demos
- ☐ Open to work settings configured

3. LeetCode: verify count; redo 5 oldest easy problems for speed.

Where to practice

- GitHub profile: <https://github.com/settings/profile>
- LinkedIn profile strength meter

Time estimate: 8 hours

WEEK 25-26 – Phase 8: Interview execution (Days 169-182) →

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Master guide: [19-phases-678-easy-to-hard-complete.pdf](#) §Phase 8

Sub	Days	Difficulty	Activity
8A	169-171	 Easy	Weak area audit, resume, GitHub, Tier 1-3 Q&A review
8B	172-174	 Medium	Partial mocks: 1 DSA (35m), 1 LLD (45m), 1 easy HLD (45m)
8C	175-177	 Med-Hard	Combined: DSA+HLD back-to-back, behavioral 4 STAR stories
8D	178-180	 Hard	Full loop sim: 2 DSA + 1 HLD + behavioral (3 hrs/day)
8E	181-182	 Hard	Apply 8 companies, Tier 5 salary Q&A, negotiation scripts

Day 169 – Phase 8A: Weak area audit

Review Phase 6 and Phase 7 gate exam failures. List top 3 weak patterns and top 3 weak HLD designs. Schedule redo.

Day 172 – Phase 8B: Partial mock #1 (1 medium DSA, 35 min timed)

Pick unseen medium from: LC 15, LC 207, LC 322. No hints first 20 min.

Day 175 – Phase 8C: Combined mock (DSA 35 min + HLD Notification 45 min)

No break between rounds — simulate onsite fatigue.

Day 178 – Phase 8D: Full loop simulation

Round 1: DSA medium (35 min) → Round 2: DSA medium (35 min) → Round 3: HLD Chat or Feed (60 min).

Day 181 – Phase 8E: Apply to 8 target companies

Adobe, Salesforce, Google, Microsoft, Amazon, Atlassian, Uber, Goldman Sachs — tailor resume bullet per company.

Ongoing Days 169–182: 2 LC/day maintenance minimum.

| 176–178 | Weak pattern revision from tracker | | 179–180 | Offer negotiation research | | 181–182 | Continue interviews until offer |

Month 6 Completion Checklist

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Milestone	Done
10 HLD systems designed on paper	☐
URL Shortener blind in 45 min	☐
8 STAR stories memorized	☐
Application tracker 5+ companies	☐
Full mock completed	☐
GitHub + LinkedIn audit	☐
220+ LeetCode	☐
2 deployed projects on resume	☐

Total estimated time: ~80–96 hours (Days 141–168)

Primary resources:

- Theory: [10-system-design-complete.md](#)
- LeetCode: <https://leetcode.com/problemset/>
- System design primer: <https://github.com/donnemartin/system-design-primer>